

AI BlogNest: AI-Powered Blog Management REST API

Project Description:

AI BlogNest harnesses Generative AI to simplify blog creation and management, offering users a secure, fast, and intelligent content-authoring experience. The platform includes a Blog Management module, which provides full CRUD operations (create, read, update, delete) over user-owned blog posts, an AI Blog Generator, which produces a complete, ready-to-publish blog post from a topic and category, and an AI Summarizer, which condenses existing blog content into a concise summary.

Utilizing Google's Gemini API through the @google/generative-ai SDK, AI BlogNest processes user inputs to deliver fast, high-quality written content. Built with Node.js and Express.js and backed by a PostgreSQL relational database, the platform ensures a secure and reliable experience. With JWT-based authentication, bcrypt.js password hashing, and responsible input validation via express-validator, AI BlogNest empowers content creators, bloggers, and small publishing teams to manage and generate blog content with confidence.

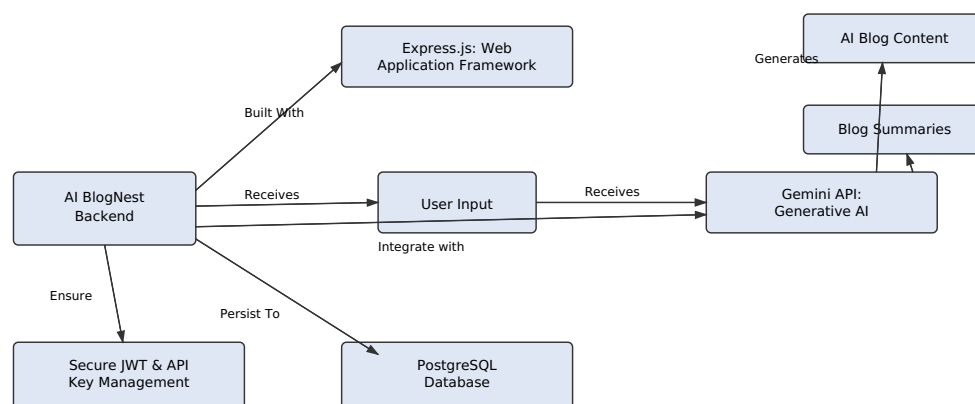
Scenarios

Scenario 1: A content creator registers an account and logs in to receive a JWT token. They create a new blog post titled "My First PostgreSQL Blog" under the Technology category. The system stores the blog against their user ID and immediately returns it, along with author details, in the API response.

Scenario 2: A blogger who is short on time provides the topic "Benefits of Data Analytics for Businesses" and category "Technology" to the AI Blog Generator. Within seconds, Gemini AI produces a complete, well-structured blog post, which is automatically saved to the blogs table and returned as JSON.

Scenario 3: A publisher has a long block of existing content and wants a short summary for a newsletter teaser. They submit the content to the AI Summarizer endpoint, and Gemini AI returns a concise, accurate summary in a fraction of the time manual summarizing would take.

Technical Architecture:



Description: AI BlogNest utilizes a modular layered architecture to deliver secure, reliable blog management with AI-assisted content generation. Express.js handles routing and request parsing, while dedicated controllers orchestrate authentication, blog CRUD, and AI operations. The backend interfaces with the Google Gemini API using the @google/generative-ai SDK to generate tailored blog content, and persists all user and blog data in a PostgreSQL database accessed through a connection pool, with secure deployment managed via Node.js and environment-based configuration.

(Note: Since AI BlogNest uses PostgreSQL as its primary database, the Entity-Relationship Diagram is included below along with a description, as required for database-backed projects.)

```

users_table.sql
1 CREATE TABLE users (
2   id SERIAL PRIMARY KEY,
3   name VARCHAR(100) NOT NULL,
4   email VARCHAR(255) UNIQUE NOT NULL,
5   password VARCHAR(255) NOT NULL,
6   created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
7   updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
8 );
9

```

users table — primary key id, unique email, hashed password

```
blogs_table.sql
1 CREATE TABLE blogs (
2   id SERIAL PRIMARY KEY,
3   title VARCHAR(255) NOT NULL,
4   content TEXT NOT NULL,
5   category VARCHAR(100) NOT NULL,
6   author INTEGER NOT NULL,
7   author_name VARCHAR(100) NOT NULL,
8   created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
9   updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
10
11   CONSTRAINT fk_blog_author
12     FOREIGN KEY (author)
13     REFERENCES users(id)
14     ON DELETE CASCADE
15 );
16
```

blogs table — foreign key author → users.id (ON DELETE CASCADE)

Pre-requisites:

- **Express.js Framework Knowledge:** [Express.js Documentation](#)
- **Google Gemini API Familiarity:** [Gemini API Documentation](#)
- **PostgreSQL & SQL Proficiency:** [PostgreSQL Documentation](#)
- **Node.js Programming Proficiency:** [Node.js Documentation](#)
- **Version Control with Git:** [Git Documentation](#)
- **Development Environment Setup:** [Node.js Installation Guide](#)
- **Postman for API Testing:** [Postman Documentation](#)

Project Workflow:

Milestone 1: Architecture and Database Design

- **Activity 1.1:** Generate a Google Gemini API key and configure it securely in the project environment.
- **Activity 1.2:** Research and select PostgreSQL as the relational database and design the users/blogs schema.
- **Activity 1.3:** Define the layered architecture of the application, detailing interactions between routes, middleware, controllers, and the AI service.
- **Activity 1.4:** Set up the development environment, installing necessary libraries and dependencies for Express and the Gemini SDK.

Milestone 2: Core Functionalities Development

- **Activity 2.1:** Develop the core functionalities: user authentication, blog CRUD, AI blog generation, and AI summarization.
- **Activity 2.2:** Implement the Express backend to manage routing, JWT verification, and request validation, ensuring smooth API interactions.

Milestone 3: Server.js & App.js Development

- **Activity 3.1:** Write the main application logic in server.js and app.js, establishing routes for each feature and integrating middleware.

Milestone 4: Database Configuration

- **Activity 4.1:** Configure the PostgreSQL connection pool and create the users and blogs tables.
- **Activity 4.2:** Implement parameterized queries across controllers to prevent SQL injection.

Milestone 5: Deployment & API Testing

- **Activity 5.1:** Prepare the application for local deployment by configuring the server environment and ensuring all dependencies are correctly installed.
- **Activity 5.2:** Test every endpoint using Postman, covering authentication, blog CRUD, and AI generation/summarization.

Milestone 6: Conclusion

Milestone 1: Architecture and Database Design

In this milestone, we focus on selecting the appropriate generative AI model and database technology for AI BlogNest's content generation and storage needs. This involves researching PostgreSQL's relational capabilities and Gemini's content-generation performance, ensuring both align with the application's objectives of secure blog management and AI-assisted writing.

Activity 1.1: Generate a Gemini API Key

Before the application can communicate with the Gemini LLM, you need a valid Gemini API key. Follow the steps below to create one and connect it securely to the project.

- **Step 1 — Create a Google AI Studio Account:** Visit <https://aistudio.google.com> and sign in with your Google account.
- **Step 2 — Navigate to API Keys:** Click "Get API Key" from the left navigation menu and select "Create API Key".
- **Step 3 — Create a New API Key:** Give your key a descriptive name (e.g., blognest-key) so it is easy to identify later.
- **Step 4 — Copy the Key:** Copy the generated key immediately and store it in a safe place, as it will not be shown again.
- **Step 5 — Add the Key to Your Project:** Create a .env file in the root of your project directory and paste the key as shown below:

```
GEMINI_API_KEY=your_copied_api_key_here
DATABASE_URL=postgresql://user:password@localhost:5432/blognest
JWT_SECRET=your_jwt_secret_here
```

- **Step 6 — Verify the Key is Loaded:** The geminiService.js file loads this key automatically at startup using dotenv:

```
const dotenv = require('dotenv');
dotenv.config();
const genAI = new GoogleGenerativeAI(process.env.GEMINI_API_KEY);
```

- **Important — Never Commit Your API Key:** Add .env to your .gitignore file to prevent accidentally pushing the key to a public repository.

Activity 1.2: Research and Select the Database and AI Model

Here are the things that needed to be researched to select the best technologies for the project:

- **Understand the Project Requirements:** Review the specific needs of AI BlogNest, focusing on secure multi-user blog storage and AI-assisted content generation.
- **Explore PostgreSQL vs. NoSQL:** Compare relational integrity (foreign keys, constraints) against the flexibility of the original MongoDB prototype, choosing PostgreSQL for stronger relational guarantees between users and blogs.
- **Evaluate Gemini Model Performance:** Compare Gemini models based on response quality, generation speed, and relevance to long-form blog writing.
- **Select the Optimal Model:** Choose the Gemini generative model for its balance of writing quality and generation speed for both full blog generation and summarization tasks.

Activity 1.3: Define the Architecture of the Application

- **Draft an Architectural Diagram:** Create a visual representation of the application architecture, including the Express server, middleware, controllers, PostgreSQL, and Gemini API integration.
- **Detail Client Interaction:** Outline how a client (Postman or a frontend) sends requests with a JSON body, platform-appropriate headers, and an Authorization Bearer token where required.
- **Outline Backend Responsibilities:** Specify how the Express backend handles incoming requests to /api/auth, /api/blogs and /api/ai, validates and sanitizes user input, and communicates with PostgreSQL and Gemini.
- **Describe AI Integration Points:** Define how aiController.js constructs prompts, calls geminiService.js, and returns structured JSON blog content or summaries to the client.

Activity 1.4: Set Up the Development Environment

- **Install Node.js and npm:** Ensure Node.js 16+ is installed on your machine along with npm for dependency management.
- **Initialize the Project:** Set up the project folder and initialize package.json:

```
D:\projects> npm init -y
```

- **Install Core Dependencies:**

```
D:\projects> npm install express cors morgan dotenv express-validator bcryptjs jsonwebtoken pg @google/generative-ai axios
```

- **Install Nodemon (Development):**

```
D:\projects> npm install nodemon --save-dev
```

- **Set Up Application Structure:** Create the project directory structure including folders for config, controllers, middleware, routes, and services.

AI BLOGNEST

■ node_modules

■ src

■ config

■ controllers

■ middleware

■ routes

■ services

■ app.js

■ server.js

■ .env

■ .gitignore

■ package-lock.json

■ package.json

■ README.md

Milestone 2: Core Functionalities Development

Activity 2.1: Develop Core Content and Blog Management Features

Implement the four core feature groups — Authentication, Blog CRUD, AI Generation, and AI Summarization — as clearly separated controllers driven by dedicated route groups.

- **Auth Controller:** Handles registration (bcrypt password hashing), login (JWT issuance), and authenticated profile retrieval.
- **Blog Controller:** Handles create, read (all/one), update, and delete operations, each scoped to the owning user for write access.
- **AI Controller:** Accepts a topic + category and returns a Gemini-generated blog, or accepts existing content and returns a Gemini-generated summary.

Activity 2.2: Implement the Express Backend

Define Routes in Express:

- Set up routes in app.js for the auth, blog, and AI endpoints, each mapped to its own router file.

Process User Input:

- Use Express's `express.json()` middleware to parse JSON bodies submitted by the client.
- Validate registration and blog fields using `express-validator` before controller logic executes.
- Validate that topic/content fields required by AI endpoints are non-empty before calling Gemini.

Integrate API Calls:

- Within the `/api/ai/generate-blog` handler, call `geminiService.js`'s `callGemini()` function to construct and submit a structured prompt.
- Submit the prompt to the Gemini API using the `@google/generative-ai` SDK.
- Parse and validate the generated text before persisting it to the blogs table and returning it to the client.

Milestone 3: Server.js & App.js Development

Milestone 3 focuses on creating and refining the core logic of the server.js and app.js files, which serve as the backbone of the AI BlogNest application. In this milestone, each feature's functionality is wired up through Express routes, with reliable middleware ordering and centralized error handling.

Activity 3.1: Writing the Main Application Logic

Define the Entry Point in server.js:

```
server.js
src > server.js > ...
1  const dotenv = require('dotenv');
2  const app = require('./app');
3  const { pool, connectDB } = require('./config/db');
4
5  dotenv.config();
6  connectDB();
7
8  const PORT = process.env.PORT || 5000;
9  app.listen(PORT, () => {
10    console.log(`Server running on port ${PORT}`);
11  });
12
```

Fig 3.1 — src/server.js

Configure the Express Application in app.js:

```
app.js
src > app.js >
1  const express = require('express');
2  const cors = require('cors');
3  const morgan = require('morgan');
4  const authRoutes = require('./routes/authRoutes');
5  const blogRoutes = require('./routes/blogRoutes');
6  const aiRoutes = require('./routes/aiRoutes');
7  const errorMiddleware = require('./middleware/errorMiddleware');
8
9  const app = express();
10
11  app.use(cors());
12  app.use(express.json());
13  app.use(morgan('dev'));
14
15  app.use('/api/auth', authRoutes);
16  app.use('/api/blogs', blogRoutes);
17  app.use('/api/ai', aiRoutes);
18
19  app.use(errorMiddleware);
20
21  app.get('/', (req, res) => {
22    res.json({ message: 'AI BlogNest API is running' });
23  });
24
25  module.exports = app;
26
```

Fig 3.2 — src/app.js

Set Up Route Handlers for Each Feature:

- For the /api/blogs routes, implement logic that reads title, content, and category from the incoming JSON body.

- Apply input validation: return a 422 error if required fields are missing.
- Route requests from Postman to the appropriate controller, which processes data and returns a JSON response.

Milestone 4: Database Configuration

Milestone 4 focuses on configuring PostgreSQL as the persistent data store for AI BlogNest, replacing the original MongoDB/Mongoose prototype with a fully relational schema.

Activity 4.1: Configure the PostgreSQL Connection

```
db.js
src > config > db.js >
1  const { Pool } = require('pg');
2
3  const pool = new Pool({
4    connectionString: process.env.DATABASE_URL,
5  });
6
7  const connectDB = async () => {
8    try {
9      await pool.query('SELECT NOW();');
10     console.log('PostgreSQL connected successfully');
11   } catch (error) {
12     console.error(`PostgreSQL connection error: ${error.message}`);
13     process.exit(1);
14   }
15 };
16
17 module.exports = { pool, connectDB };
18
```

Fig 4.1 — src/config/db.js

```
PostgreSQL connected successfully
```

Activity 4.2: Define the Database Schema

users table:

```
users_table.sql
src > config > schema.sql > ...
1  CREATE TABLE users (
2    id SERIAL PRIMARY KEY,
3    name VARCHAR(100) NOT NULL,
4    email VARCHAR(255) UNIQUE NOT NULL,
5    password VARCHAR(255) NOT NULL,
6    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
7    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
8  );
9
```

blogs table:

blogs_table.sql

```
src > config > schema.sql >
1 CREATE TABLE blogs (
2     id SERIAL PRIMARY KEY,
3     title VARCHAR(255) NOT NULL,
4     content TEXT NOT NULL,
5     category VARCHAR(100) NOT NULL,
6     author INTEGER NOT NULL,
7     author_name VARCHAR(100) NOT NULL,
8     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
9     updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
10
11     CONSTRAINT fk_blog_author
12         FOREIGN KEY (author)
13         REFERENCES users(id)
14         ON DELETE CASCADE
15 );
16
```

Activity 4.3: Apply Parameterized Queries

All controllers use parameterized SQL to prevent injection, for example:

```
pool.query('SELECT id FROM users WHERE email = $1', [email]);
```

Milestone 5: Deployment & API Testing

In Milestone 5, the focus is on deploying AI BlogNest locally and verifying every endpoint using Postman before the final demonstration.

Activity 5.1: Preparing the Application for Local Deployment

```
D:\projects> npm install
D:\projects> npm run dev
```

Expected output:

```
Server running on port 5000
PostgreSQL connected successfully
```

Activity 5.2: API Testing with Postman

1. User Registration — POST <http://localhost:5000/api/auth/register>

POST

http://localhost:5000/api/auth/register

Send

Body

Status: 201 Created Time: 140 ms Size: 397 B

```
1 {
2   "name": "Test User",
3   "email": "testuser@gmail.com",
4   "password": "test123456"
5 }
```

```
{
  "message": "User registered successfully",
  "user": {
    "id": 1,
    "name": "Test User",
    "email": "testuser@gmail.com"
  }
}
```

2. User Login — POST <http://localhost:5000/api/auth/login>

POST

http://localhost:5000/api/auth/login

Send

Body

Status: 200 OK Time: 146 ms Size: 451 B

```
1 {
2   "email": "testuser@gmail.com",
3   "password": "test123456"
4 }
```

Do not include your actual JWT token in the documentation.

```
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6MX0uR7c0dn0D0_ev8Ss57k"
}
```

3. Create Blog — POST <http://localhost:5000/api/blogs> (Authorization: Bearer Token)

The screenshot displays a REST client interface with a dark theme. At the top, a 'POST' button is visible next to the URL 'http://localhost:5000/api/blogs'. A 'Send' button is located on the right. The main area is divided into two panels: 'Body' on the left and 'Status' on the right. The 'Body' panel shows the request JSON, and the 'Status' panel shows the response status and body.

```
POST http://localhost:5000/api/blogs

{
  "title": "My First PostgreSQL Blog",
  "content": "This is my first blog created using the AI BlogNest API with PostgreSQL.",
  "category": "Technology"
}
```

Status: 201 Created Time: 59 ms Size: 541 B

```
{
  "id": 1,
  "title": "My First PostgreSQL Blog",
  "content": "This is my first blog created using the AI BlogNest API with PostgreSQL.",
  "category": "Technology",
  "author": 1,
  "authorName": "Test User",
  "createdAt": "2026-09-25T06:43:11.781Z",
  "updatedAt": "2026-09-25T06:43:11.781Z"
}
```

4. Generate AI Blog — POST <http://localhost:5000/api/ai/generate-blog> (Authorization: Bearer Token)

POST http://localhost:5000/api/ai/generate-blog Send	
Body	Status: 201 Created Time: 4.12 s Size: 1.26 KB
<pre>1 { 2 "topic": "Benefits of Data Analytics for Businesses", 3 "category": "Technology" 4 }</pre>	<pre>{ "id": 2, "title": "Benefits of Data Analytics for Businesses", "content": "Data analytics has become a cornerstone of modern business strategy, helping organizations uncover patterns, forecast trends and make evidence-based decisions...", "category": "Technology", "author": 1, "authorName": "Test User" }</pre>

5. Summarize Blog — POST http://localhost:5000/api/ai/summarize

POST http://localhost:5000/api/ai/summarize Send	
Body	Status: 200 OK Time: 5.67 s Size: 702 B
<pre>1 { 2 "content": "Data analytics helps businesses make better decisions by using data to identify trends, understand customers, improve operations, and reduce risks." 3 } 4 5</pre> The exact generated summary may vary because it is produced by Gemini AI.	<pre>{ "summary": "Data analytics helps businesses make better decisions by identifying trends, understanding customers, improving operations, and reducing risks." }</pre>

Milestone 6: Conclusion

With deployment and testing complete, the project moves into final documentation and demonstration preparation, covered in the sections below.

Exploring the API Endpoints:

Authentication Endpoints:

Description: The authentication group exposes `/api/auth/register`, `/api/auth/login`, and `/api/auth/profile`. It handles account creation with bcrypt-hashed passwords, JWT issuance on login, and authenticated profile retrieval using a Bearer token — forming the security foundation for every protected route in the application.

GET	http://localhost:5000/api/auth/profile	Send
Body	Status: 200 OK Time: 64 ms Size: 118 B	
(no body)	<pre>{ "id": 1, "name": "Test User", "email": "testuser@gmail.com" }</pre>	

Blog Management Endpoints:

Description: The blog group exposes `/api/blogs` and `/api/blogs/:id`, supporting full CRUD. Public GET requests allow anyone to browse published blogs, while POST, PUT, and DELETE are protected by JWT and scoped to the requesting user's own posts.

GET	http://localhost:5000/api/blogs	Send
Body	Status: 200 OK Time: 92 ms Size: 587 B	
(no body)	<pre>[{ "id": 1, "title": "My First PostgreSQL Blog", "content": "This is my first blog created using the AI BlogNest API with PostgreSQL.", "category": "Technology", "author": { "id": 1, "name": "Test User", "email": "testuser@gmail.com" }, "authorName": "Test User", "createdAt": "...", "updatedAt": "..." }]</pre>	

AI Endpoints:

Description: The AI group exposes `/api/ai/generate-blog` and `/api/ai/summarize`. These routes call the Gemini service to synthesize a full blog post from a topic/category, or to condense existing content into a short summary, returning structured JSON to the client in both cases.

GET	http://localhost:5000/	Send
Body	Status: 200 OK Time: 12 ms Size: 46 B	
(no body)	<pre>{ "message": "AI BlogNest API is running" }</pre>	

Conclusion:

AI BlogNest is a secure, modular REST API built with Node.js, Express.js, and PostgreSQL, and enhanced with Google Gemini AI for content generation and summarization. It uses JWT authentication and bcrypt.js password hashing to protect user-specific operations, while a clean layered architecture (routes → middleware → controllers → services) keeps AI integration decoupled from core CRUD logic. The project, which included database migration from MongoDB to PostgreSQL, development, and thorough Postman-based testing, highlights how relational data modeling and targeted generative AI can boost productivity for bloggers and content teams, with clear potential for future scalability. Looking ahead, AI BlogNest offers significant opportunities for expansion, such as adding role-based access control, introducing a caching layer for high-traffic reads, and queuing AI requests asynchronously. By continuously refining the Gemini prompts and hardening the API surface, the platform is well-positioned to evolve from a focused backend service into a comprehensive AI-powered content management system.